

HOME AUTOMATION SYSTEM

Complete Project Documentation

ESP32-Based Master-Slave Home Automation System with Firebase Cloud Integration

Status: Production Ready

Project Completion Date: July 7, 2026

Table of Contents

1. Project Overview

Abstract

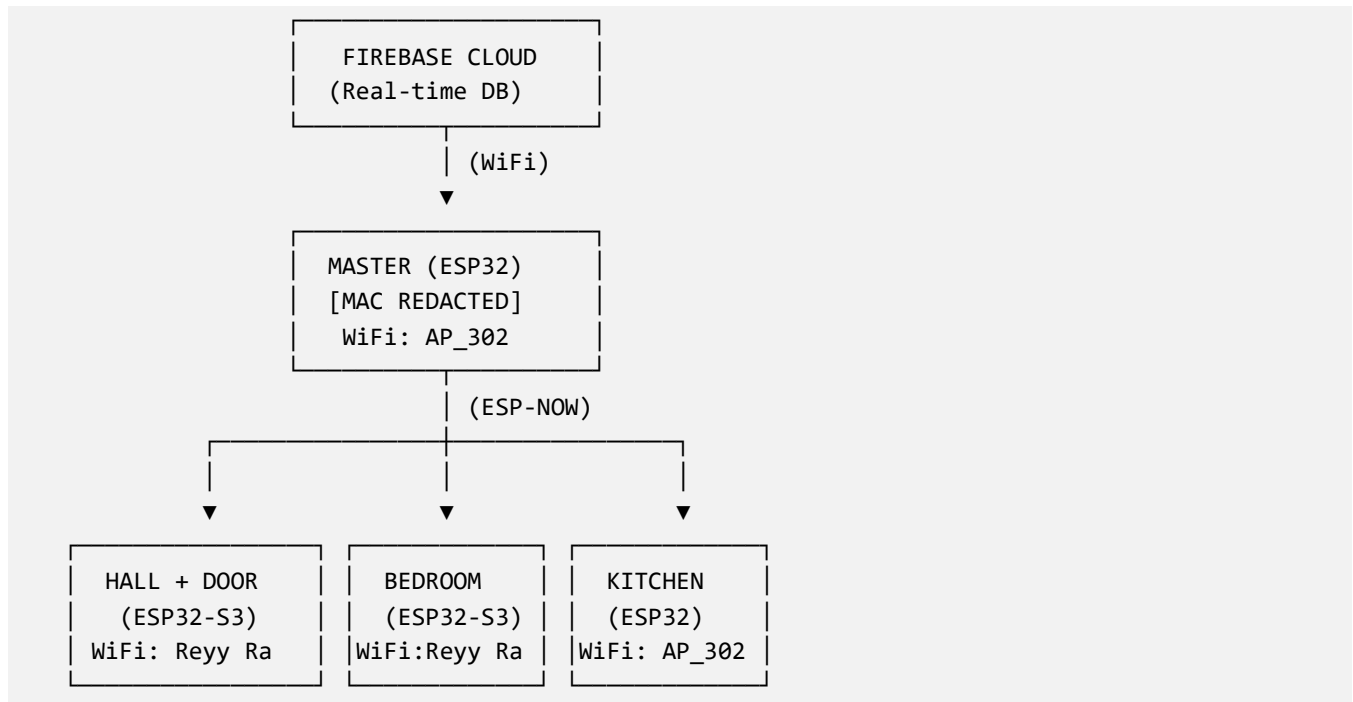
This project presents a comprehensive home automation system using ESP32 microcontrollers in a master-slave architecture. The system enables remote monitoring and control of multiple rooms (Hall, Bedroom, Kitchen, and Door) through a centralized master unit connected to Firebase Cloud. Features include real-time sensor data acquisition, automated appliance control based on environmental conditions, manual control through touch interfaces, and cloud-based web dashboard access.

Key Features

- Master-Slave Architecture using ESP-NOW protocol
- Firebase Cloud Integration for remote monitoring and control
- Real-time Sensor Monitoring (Temperature, Motion, Light, Gas)
- Automated Appliance Control (Light, Fan, Exhaust, Door Lock)
- Manual Touch Controls for local operation
- State Persistence via Flash Memory
- Safety Protocols (Gas detection with automatic exhaust)
- Scalable Design supporting multiple rooms

2. System Architecture

2.1 Network Topology



2.2 Communication Protocols

Protocol	Purpose	Devices
ESP-NOW	Master-Slave Communication	All ESP32 devices

Protocol	Purpose	Devices
WiFi	Cloud Connectivity	Master ESP32
I2C	Sensor Communication	MCP9808 (Bedroom)
PWM	Fan Speed Control	Bedroom Fan

2.3 Hardware Components

Microcontrollers

Device	Quantity	Purpose
ESP32	1	Master Controller
ESP32-S3	2	Hall+Door & Bedroom
ESP32	1	Kitchen Controller

Sensors

Sensor	Purpose	Room
MCP9808 (I2C)	Temperature	Bedroom
PIR Motion	Motion Detection	Bedroom
IR Obstacle	Motion Detection	Hall
LDR	Light Detection	Hall
MQ-2	Gas Detection	Kitchen

Actuators

Component	Purpose	Room
Relays	Appliance Control	All Rooms
Solenoid Lock	Door Lock	Door
Buzzer	Alert/Feedback	Hall, Kitchen
PWM Fan	Ventilation	Bedroom, Hall
Touch Sensors	Manual Control	All Rooms

3. Pin Configuration

3.1 Hall + Door (ESP32-S3)

LIGHT_PIN	37	Fan Relay
FAN_PIN	38	Fan Relay
TOUCH_LIGHT_PIN	39	Touch Control
TOUCH_FAN_PIN	40	Touch Control
IR_PIN	41	Motion Sensor
LDR_PIN	3	Light Sensor
BUZZER_PIN	9	Alert
SOLENOID_PIN	1	Door Lock
DOOR_TOUCH_PIN	2	Door Touch

3.2 Bedroom (ESP32-S3)

LED_PIN	3	Light Relay
FAN_PWM	9	PWM Fan Control
TOUCH_LED	10	Touch Control
TOUCH_FAN	37	Touch Control
PIR_PIN	38	Motion Sensor
SDA_PIN	39	I2C Data
SCL_PIN	40	I2C Clock

3.3 Kitchen (ESP32)

LIGHT_PIN	45	Light Relay
EXHAUST_PIN	48	Exhaust Relay
TOUCH_LIGHT_PIN	47	Touch Control
TOUCH_EXHST_PIN	21	Touch Control
MQ2_DOUT_PIN	13	Gas Sensor

3.4 Master (ESP32)

WiFi SSID: AP_302

WiFi Password: [REDACTED - see credentials note, Section 5.2]

MAC: [REDACTED]

4. Features by Room

4.1 Hall + Door

- Light Control: Touch + Website
- Fan Control: Touch + Website
- Auto Mode: Light turns ON with Motion + Darkness
- Motion Sensor: IR sensor with 5-second timeout
- LDR Sensor: Ambient light detection
- Buzzer: Alert/Feedback
- Door Lock: Solenoid with Touch + Website
- Flash Memory: State persistence

4.2 Bedroom

- Light Control: Touch + Website
- Fan Control: Touch + Website
- Auto Mode: Fan ON when Motion + Temp > 30°C
- Fan Speed: PWM controlled (0-100%)
 - 20-30°C → 0%
 - 30-32°C → 40%
 - 33-35°C → 70%
 - 36°C+ → 100%
- Temperature Sensor: MCP9808 (I2C)
- Motion Sensor: PIR with 30-second timeout
- Flash Memory: State persistence

4.3 Kitchen

- Light Control: Touch + Website
- Exhaust Control: Touch + Website
- Gas Detection: MQ-2 with confidence filter (3 readings)
- Safety: Automatic Exhaust ON when gas detected
- Safety Lock: Blocks Exhaust OFF when gas present
- Buzzer: Gas alert
- Auto Exhaust: Turns OFF when gas clears

5. Firebase Data Structure

5.1 Database Schema

```

/firebase
├── /hall
│   ├── light (bool)
│   ├── fan (bool)
│   ├── autoMode (bool)
│   ├── motion (bool)
│   ├── ldr (int)
│   └── buzzer (bool)
├── /bedroom
│   ├── bedLight (bool)
│   ├── fan (bool)
│   ├── autoMode (bool)
│   ├── temp (float)
│   ├── fanSpeed (int)
│   └── motion (bool)
├── /kitchen
│   ├── light (bool)
│   ├── exhaustFan (bool)
│   ├── gas (bool)
│   └── buzzer (bool)
├── /door
│   ├── locked (bool)
│   ├── streamUrl (string)
│   └── camera (bool)
├── /alerts
│   └── gasAlert (bool)
├── /doorLogs
│   └── logs (array)
└── /master
    ├── online (bool)
    ├── wifi (int)
    ├── espnow (int)
    └── roomOnline (bool)

```

5.2 Firebase Configuration

⚠ Security note: *Live credentials (password, full API key/DB URL) have been redacted from this copy. Keep real values in an environment file (e.g. .env) excluded from version control and any shared document — never in plaintext documentation.*

```
API_KEY       : [REDACTED]
DATABASE_URL  : [REDACTED]
USER_EMAIL    : [REDACTED]
USER_PASSWORD : [REDACTED]
CAM_STREAM_URL : http://[LOCAL_IP]:8080/video
```

6. Command Set

6.1 Hall Commands

```
LED_ON / LED_OFF    - Light Control
FAN_ON / FAN_OFF    - Fan Control
AUTO_ON / AUTO_OFF  - Auto Mode
BUZZER_ON / BUZZER_OFF - Buzzer Control
STATUS              - Request Status
```

6.2 Bedroom Commands

```
LED_ON / LED_OFF    - Light Control
FAN_ON / FAN_OFF    - Fan Control
FAN_LOW / FAN_HIGH  - Fan Speed
AUTO_ON / AUTO_OFF  - Auto Mode
STATUS              - Request Status
```

6.3 Kitchen Commands

```
LED_ON / LED_OFF    - Light Control
EXHAUST_ON / EXHAUST_OFF - Exhaust Control
STATUS              - Request Status
```

6.4 Door Commands

```
LOCK / UNLOCK       - Door Lock Control
STATUS              - Request Status
```

7. State Management

7.1 Flash Memory Storage (Preferences)

Room	Stored Variables
Hall	lightOn, fanOn, autoMode, buzzerActive
Bedroom	light, fan, auto
Kitchen	(states restored on boot)
Door	isLocked

7.2 State Change Prevention (Echo Guard)

```
#define LOCAL_GUARD_MS 3000
// Prevents processing own echoes for 3 seconds
```

8. Safety Features

8.1 Kitchen Gas Safety

- Gas detection with confidence filtering
- Automatic exhaust ON when gas confirmed
- Blocking exhaust OFF command when gas present
- Buzzer alert on gas detection
- 30-second warm-up period to prevent false alarms

8.2 Bedroom Auto Mode

- Fan only turns ON with motion + temperature
- Speed adjusts based on temperature
- Fan OFF when no motion (30-second timeout)
- Manual control blocks when auto mode active

8.3 Hall Auto Mode

- Light ON with motion + darkness
- Light OFF after 5 seconds of no motion
- LDR recovery delay (15 seconds)

8.4 System Protection

- Boot ignore period (5-6 seconds)
- Debounced touch inputs (50-500ms)
- ESP-NOW retry mechanism
- Status transmission throttling (200ms)
- I2C timeout protection (100ms)

9. Timing Parameters

9.1 Sensor Intervals

Parameter	Value	Purpose
SENSOR_INTERVAL	2s	Sensor reading
STATUS_INTERVAL	5s	Status update
TEMP_READ_INTERVAL	4s	Temperature reading
GAS_CHECK_MS	200ms	Gas monitoring
HELLO_INTERVAL	5s	Master handshake

9.2 Timeouts

Parameter	Value	Purpose
MOTION_TIMEOUT	30s	No motion clear
PIR_CLEAR_MS	3s	Motion confirm
LDR_RECOVERY	15s	Light recovery
BOOT_IGNORE	5-6s	Start-up protection

9.3 Debounce

Parameter	Value	Purpose
TOUCH_DEBOUNCE	50-500ms	Touch filtering
IR_DEBOUNCE	150ms	IR filtering

10. Technical Specifications

10.1 Communication

- ESP-NOW: 2.4GHz, 1-11 channels
- WiFi: 2.4GHz, 802.11 b/g/n
- Firebase: Realtime Database (RTDB)
- Serial: 115200 baud

10.2 Power

- ESP32: 5V USB, ~80mA (active)
- Relays: 5V, ~70mA per relay
- Sensors: 3.3V/5V, low power

10.3 Performance

- Response Time: <100ms (ESP-NOW)
- Update Rate: 5s (periodic status)
- Sensor Read: 2-4s intervals
- Streaming: Real-time Firebase

11. Installation Guide

11.1 Prerequisites

- Arduino IDE (v1.8.19 or later)
- ESP32 Board Package (v2.0.14 or later)

Required Libraries:

```
- WiFi.h (built-in)
- esp_now.h (built-in)
- Firebase_ESP_Client.h
- Adafruit_MCP9808.h
- Preferences.h (built-in)
- Wire.h (built-in)
```

11.2 Library Installation

1. Open Arduino IDE
2. Go to Sketch → Include Library → Manage Libraries
3. Install:
 - "Firebase ESP Client" by Mobizt
 - "Adafruit MCP9808" by Adafruit
 - "ESP32" board package via Boards Manager

11.3 Upload Steps

1. Master Upload

- Select Board: DOIT ESP32 DEVKIT V1
- Upload Master_Code.ino
- Verify Serial Monitor output

2. Hall+Door Upload

- Select Board: ESP32S3 Dev Module
- Upload Hall_Door_Slave.ino
- Verify Serial Monitor output

3. Bedroom Upload

- Select Board: ESP32S3 Dev Module
- Upload Bedroom_Slave.ino
- Verify Serial Monitor output

4. Kitchen Upload

- Select Board: DOIT ESP32 DEVKIT V1
- Upload Kitchen_Slave.ino
- Verify Serial Monitor output

11.4 Wiring Connections

1. Connect all relay modules to respective ESP32 pins
2. Connect sensors (I2C, PIR, IR, LDR, MQ2) to appropriate pins
3. Power all ESP32 boards via USB
4. Ensure common ground if needed

12. Troubleshooting

12.1 Common Issues & Solutions

Problem	Solution
ESP-NOW not working	Check MAC addresses, same WiFi channel
Firebase not connecting	Verify API key, internet connection
Sensors not reading	Check wiring, I2C addresses

Problem	Solution
Touch not responding	Check INPUT_PULLDOWN configuration
Auto mode not working	Check sensor readings, conditions
State not persisting	Verify Preferences initialization
Gas false alarms	Check warm-up time, sensor placement

12.2 Debugging Tips

- Use Serial Monitor at 115200 baud
- Check MAC addresses on boot
- Monitor sensor readings periodically
- Verify Firebase streams are active
- Check command queues and pending messages

13. Future Enhancements

13.1 Planned Features

- OTA Updates for all devices
- Energy monitoring (current/voltage)
- Voice control (Alexa/Google Assistant)
- Mobile App development
- Machine learning for predictive automation
- Additional sensors (humidity, air quality)

13.2 Scalability

- Support for up to 20 slave devices
- Room addition without code changes
- Modular architecture for easy expansion
- Cross-platform web dashboard

14. Project Team

Team Members

Lead Developer: [Your Name]

- System Architecture Design
- ESP32 Firmware Development
- Firebase Integration
- Testing & Documentation

Acknowledgments

- Firebase Documentation
- ESP32 Community Forums

- [Adafruit Library Support](#)

15. References

Documentation

- [ESP-NOW](#)
- [Firebase Realtime Database](#)
- [MCP9808](#)

Libraries

- [Firebase ESP Client](#)
- [Adafruit MCP9808](#)

16. Appendix

A. Message Format

```
HELLO|RoomName|MAC  
ALERT|RoomName|KEY=VALUE;KEY=VALUE  
CMD|RoomName|COMMAND
```

B. Status Format

```
LIGHT=1;FAN=1;AUTO=1;TEMP=25.5;PIR=1;FANSPEED=40
```

C. MAC Address Format

```
XX:XX:XX:XX:XX:XX  
Example: [REDACTED]
```

17. Hardware Build Photos

Physical prototype photos of the per-room slave assemblies and the master controller board, as built and wired for testing.

17.1 Kitchen Assembly

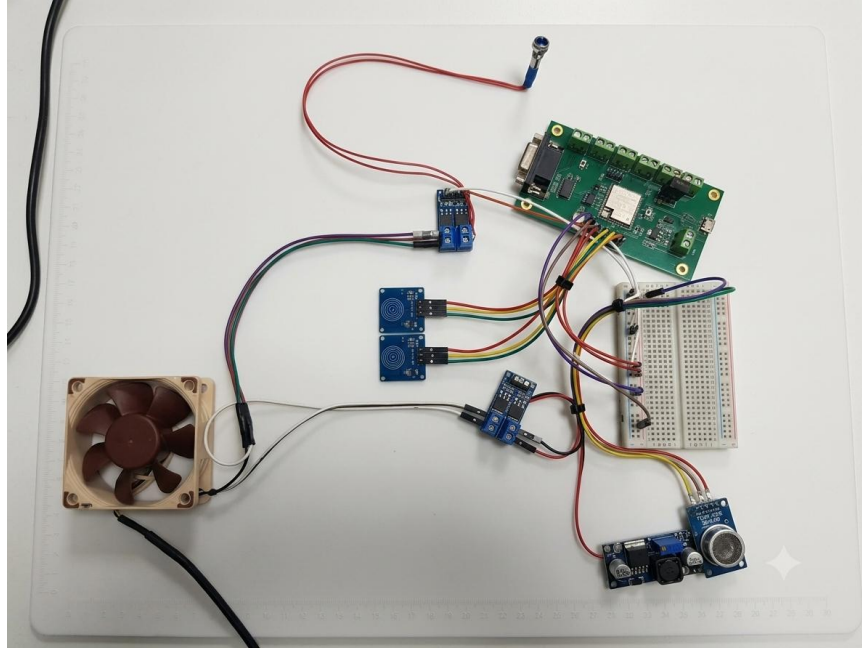


Fig. 17.1 - Kitchen breadboard prototype: relay modules, dual touch sensors, gas/driver module, fan, and ESP32 dev board.

17.2 Hall+Door Assembly

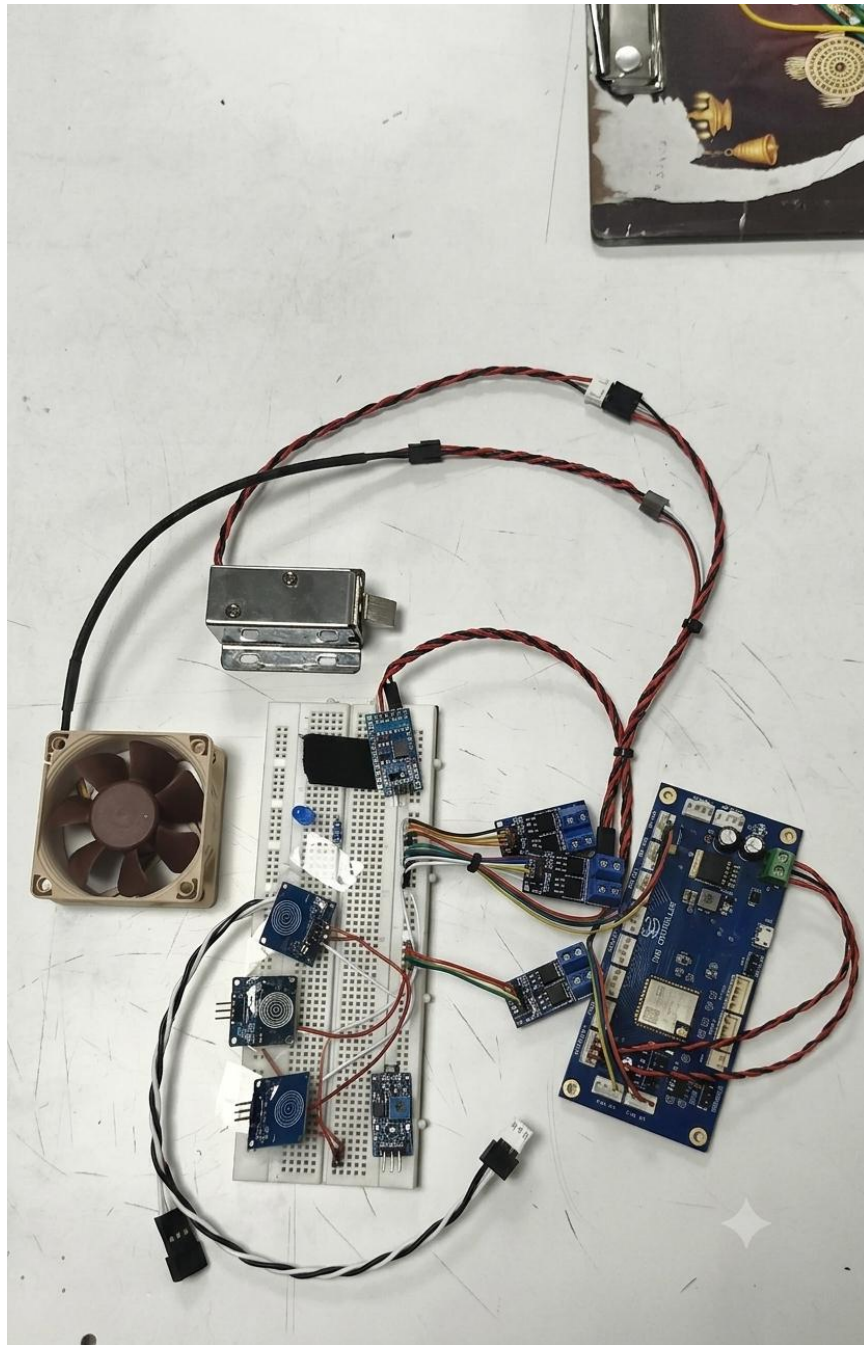


Fig. 17.2 - Kitchen assembly on breadboard, including solenoid lock, touch sensors, MQ-2 driver, fan, and ESP32 dev board.

17.3 Master Controller Board



Fig. 17.3 - ESP32 DevKit module used as the master controller.

18. Web Dashboard Screenshots

Screenshots of the web control dashboard, covering authentication, the master overview, and each room's control page.

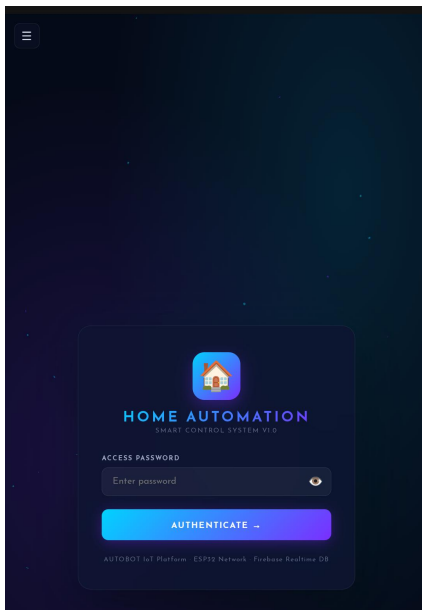


Fig. 18.1 - Login screen: password-gated access to the AUTOBOT IoT platform, confirming ESP32 network and Firebase Realtime DB connectivity.

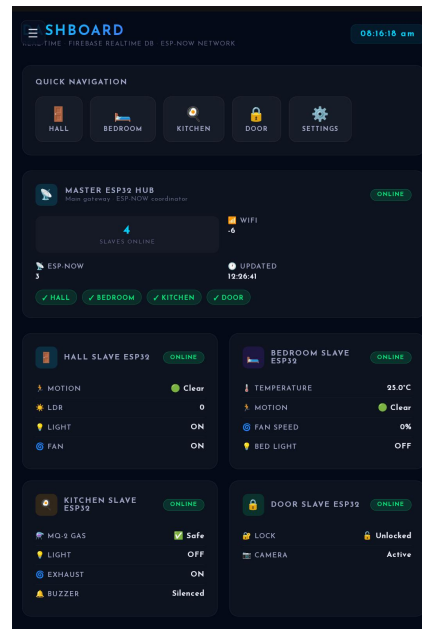


Fig. 18.2 - Dashboard: quick navigation to all rooms, master hub status (slaves online, WiFi signal, ESP-NOW peer count), and live per-room summaries.

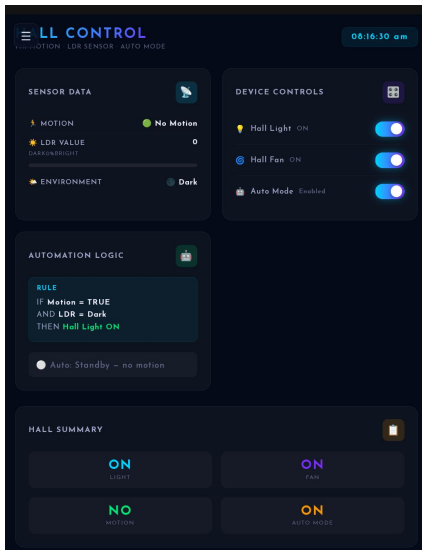


Fig. 18.3 - Hall control: motion and LDR readings, manual light/fan toggles, the auto-mode rule, and a live hall summary panel.

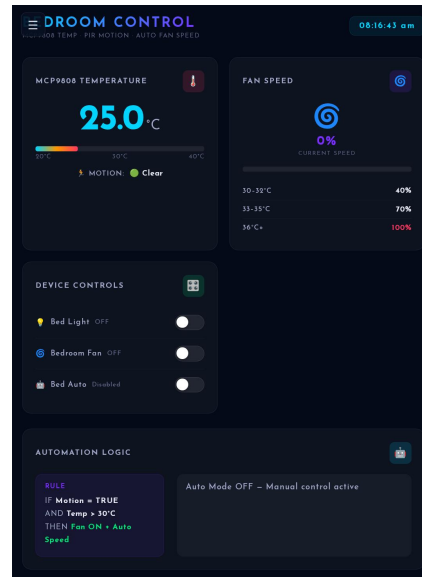


Fig. 18.4 - Bedroom control: live MCP9808 temperature, PIR motion status, PWM fan speed gauge, and manual bed light/fan/auto toggles.

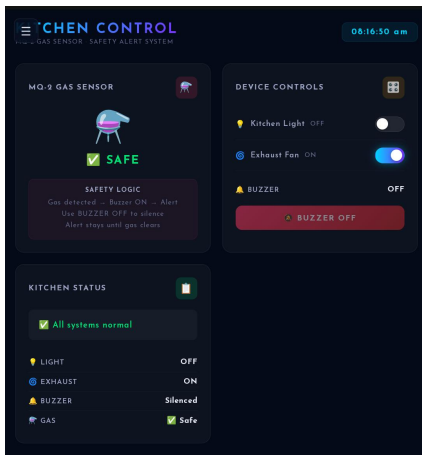


Fig. 18.5 - Kitchen control: MQ-2 gas sensor status, safety logic summary, buzzer silence control, and live kitchen system status.



Fig. 18.6 - Door security: solenoid lock state with lock/unlock controls, plus a security camera panel (offline, awaiting ESP32-CAM).

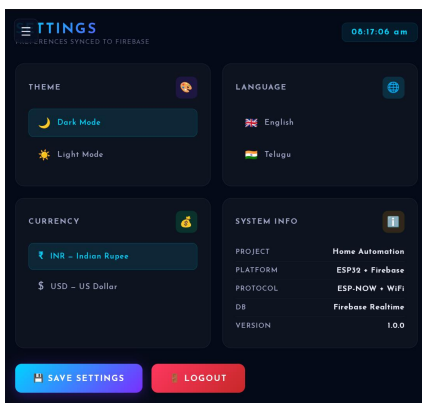


Fig. 18.7 - Settings: theme, language, and currency preferences, system info summary, and save/logout controls, synced to Firebase.

19. Appendix D: Full Slave Source Code

Security note: *WiFi passwords and master MAC addresses have been redacted below. Restore the real values only in your local, non-shared copy of the firmware.*

Note: The Kitchen and Bedroom firmware target a different master MAC address than the Hall+Door firmware and the master's documented MAC (Section 3.4). If all three slaves are meant to pair with the same physical master, this mismatch is worth double-checking -- it would explain a slave failing to reach the master over ESP-NOW even though WiFi and power are fine.

19.1 Hall + Door Slave (ESP32-S3)

```

/*
=====
Home Automation - HALL + DOOR SLAVE (ESP32-S3) - FINAL (REVISED)
=====
HALL:
  Light   : GPIO37 | Fan       : GPIO38
  TLight  : GPIO39 | TFan      : GPIO40
  IR      : GPIO41 | LDR(digital): GPIO3
  Buzzer  : GPIO9
DOOR:
  Solenoid : GPIO1 | Touch      : GPIO2
WiFi : Reyy Ra / [REDACTED]
Master: [REDACTED]
=====
*/
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>
#include <Preferences.h>

#define WIFI_SSID      "Reyy Ra"
#define WIFI_PASSWORD  "[REDACTED]"

#define LIGHT_PIN      37
#define FAN_PIN        38
#define TOUCH_LIGHT_PIN 39
#define TOUCH_FAN_PIN  40
#define IR_PIN         41
#define LDR_PIN        3
#define BUZZER_PIN     9
#define SOLENOID_PIN   1
#define DOOR_TOUCH_PIN 2

uint8_t masterMAC[] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF}; // [REDACTED]

// -- State --
bool lightOn      = false;
bool fanOn        = false;
bool autoMode     = false;
bool buzzerActive = false;
bool motionNow    = false;
bool isDark       = false;
bool isLocked     = true;

// -- Echo guards --
int  lastSentLight = -1;
int  lastSentFan   = -1;
int  lastSentLocked = 1;
unsigned long lastLocalLightChange = 0;
unsigned long lastLocalFanChange   = 0;
unsigned long lastDoorLocalChange  = 0;
#define LOCAL_GUARD_MS 3000

// -- Timers --
unsigned long lastLightOffTime = 0;
unsigned long lastMotionTime   = 0;
unsigned long lastStateChangeTime = 0;
unsigned long lastDoorTouchTime = 0;

```

```

unsigned long lastHelloTime      = 0;
unsigned long bootIgnoreUntil    = 0;

#define BOOT_IGNORE_MS          5000
#define TOUCH_DEBOUNCE_MS      500
#define DEBOUNCE_LOCKOUT_MS    500
#define MOTION_TIMEOUT_MS      5000
#define LDR_RECOVERY_MS        15000
#define HELLO_INTERVAL_MS      4000

bool masterAked = false;

char rxBuffer[128];
char processBuf[128];
volatile bool newMsgAvailable = false;

Preferences preferences;

// -----
void triggerBeep(int ms) {
    digitalWrite(BUZZER_PIN, HIGH); delay(ms); digitalWrite(BUZZER_PIN, LOW);
}

void setBuzzer(bool on) {
    digitalWrite(BUZZER_PIN, on ? HIGH : LOW);
    buzzerActive = on;
}

void setLight(bool on) {
    if (lightOn == on) return;
    lightOn = on;
    digitalWrite(LIGHT_PIN, on ? HIGH : LOW);
    lastStateChangeTime = millis();
    preferences.putBool("lightOn", lightOn);
    if (!on) lastLightOffTime = millis();
}

void setFan(bool on) {
    if (fanOn == on) return;
    fanOn = on;
    digitalWrite(FAN_PIN, on ? HIGH : LOW);
    lastStateChangeTime = millis();
    preferences.putBool("fanOn", fanOn);
}

void setLock(bool locked) {
    isLocked = locked;
    digitalWrite(SOLENOID_PIN, locked ? LOW : HIGH);
    preferences.putBool("isLocked", isLocked);
    Serial.println(locked ? "LOCKED" : "UNLOCKED");
}

void sendHallStatus() {
    String pkt = "ALERT|Hall|"
        "LIGHT=" + String(lightOn ? "1":"0") +
        ";FAN=" + String(fanOn ? "1":"0") +
        ";AUTO=" + String(autoMode ? "1":"0") +
        ";BUZZER="+ String(buzzerActive?"1":"0");
    esp_now_send(masterMAC, (uint8_t*)pkt.c_str(), pkt.length()+1);
    Serial.println("TX Hall: " + pkt.substring(11));
    lastSentLight = lightOn ? 1 : 0;
    lastSentFan = fanOn ? 1 : 0;
}

void sendDoorStatus() {
    String pkt = "ALERT|Door|LOCKED=" + String(isLocked?"1":"0") + ";CAMERA=1";
    esp_now_send(masterMAC, (uint8_t*)pkt.c_str(), pkt.length()+1);
    Serial.println("TX Door: LOCKED=" + String(isLocked?"1":"0"));
    lastSentLocked = isLocked ? 1 : 0;
}

void sendHello() {

```

```

String h = "HELLO|Hall|";
esp_now_send(masterMAC, (uint8_t*)h.c_str(), h.length()+1);
delay(50);
String d = "HELLO|Door|";
esp_now_send(masterMAC, (uint8_t*)d.c_str(), d.length()+1);
Serial.println("HELLO Hall+Door -> Master");
}

// -----
void onDataReceived(const esp_now_recv_info_t *info,
                   const uint8_t *data, int len) {
    masterAcked = true;
    int n = min(len, 127);
    memcpy(rxBuffer, data, n);
    rxBuffer[n] = '\0';
    newMsgAvailable = true;
}

void onDataSent(const esp_now_send_info_t *info, esp_now_send_status_t status) {
    // Serial.println(status==ESP_NOW_SEND_SUCCESS?"OK":"FAIL");
}

// -----
void processCommand(String msg) {
    Serial.println("RX: " + msg);

    int p1 = msg.indexOf('|'), p2 = msg.indexOf('|', p1+1);
    if (p1 < 0 || p2 < 0) return;
    String type = msg.substring(0, p1);
    String room = msg.substring(p1+1, p2);
    String cmd = msg.substring(p2+1);

    if (type != "CMD") return;

    if (millis() < bootIgnoreUntil) {
        Serial.println("Boot window - skip: " + cmd); return;
    }

    Serial.println "[" + room + "] CMD: " + cmd);

    // -- DOOR --
    if (room == "Door") {
        if (cmd == "LOCK") {
            if (millis()-lastDoorLocalChange<LOCAL_GUARD_MS && lastSentLocked==1) {
                Serial.println("Door echo ignored"); return;
            }
            setLock(true); lastSentLocked=1; sendDoorStatus();
        }
        else if (cmd == "UNLOCK") {
            if (millis()-lastDoorLocalChange<LOCAL_GUARD_MS && lastSentLocked==0) {
                Serial.println("Door echo ignored"); return;
            }
            setLock(false); lastSentLocked=0; sendDoorStatus();
        }
        else if (cmd == "STATUS") { sendDoorStatus(); }
        return;
    }

    // -- HALL --
    if (room == "Hall") {
        bool changed = false;
        if (cmd == "LED_ON") {
            if (millis()-lastLocalLightChange<LOCAL_GUARD_MS && lastSentLight==1) { Serial.println("Light echo ignored");
            return; }
            setLight(true); changed=true;
        }
        else if (cmd == "LED_OFF") {
            if (millis()-lastLocalLightChange<LOCAL_GUARD_MS && lastSentLight==0) { Serial.println("Light echo ignored");
            return; }
            setLight(false); changed=true;
        }
        else if (cmd == "FAN_ON") {
            if (millis()-lastLocalFanChange<LOCAL_GUARD_MS && lastSentFan==1) { Serial.println("Fan echo ignored"); return; }

```

```

    setFan(true); changed=true;
}
else if (cmd == "FAN_OFF") {
    if (millis()-lastLocalFanChange<LOCAL_GUARD_MS && lastSentFan==0) { Serial.println("Fan echo ignored"); return; }
    setFan(false); changed=true;
}
else if (cmd == "AUTO_ON") { autoMode=true; preferences.putBool("autoMode",true); Serial.println("Auto ON");
changed=true; }
else if (cmd == "AUTO_OFF") { autoMode=false; preferences.putBool("autoMode",false); Serial.println("Auto OFF");
changed=true; }
else if (cmd == "BUZZER_ON") { setBuzzer(true); preferences.putBool("buzzerActive",true); changed=true; }
else if (cmd == "BUZZER_OFF") { setBuzzer(false); preferences.putBool("buzzerActive",false); changed=true; }
else if (cmd == "STATUS") { sendHallStatus(); return; }
if (changed) sendHallStatus();
}
}

// -----
void setup() {
    Serial.begin(115200); delay(500);
    bootIgnoreUntil = millis() + BOOT_IGNORE_MS;
    Serial.println("HALL+DOOR SLAVE BOOTING");

    pinMode(LIGHT_PIN,      OUTPUT);
    pinMode(FAN_PIN,        OUTPUT);
    pinMode(BUZZER_PIN,     OUTPUT);
    pinMode(SOLENOID_PIN,   OUTPUT);

    pinMode(TOUCH_LIGHT_PIN, INPUT_PULLDOWN);
    pinMode(TOUCH_FAN_PIN,   INPUT_PULLDOWN);
    pinMode(DOOR_TOUCH_PIN,  INPUT_PULLDOWN);

    pinMode(IR_PIN,         INPUT);
    pinMode(LDR_PIN,        INPUT);

    preferences.begin("home-node", false);
    autoMode   = preferences.getBool("autoMode",    false);
    lightOn    = preferences.getBool("lightOn",     false);
    fanOn      = preferences.getBool("fanOn",       false);
    buzzerActive = preferences.getBool("buzzerActive", false);
    isLocked   = preferences.getBool("isLocked",    true);

    digitalWrite(LIGHT_PIN, lightOn ? HIGH : LOW);
    digitalWrite(FAN_PIN,   fanOn ? HIGH : LOW);
    digitalWrite(BUZZER_PIN, buzzerActive ? HIGH : LOW);
    setLock(isLocked);

    lastSentLight = lightOn ? 1 : 0;
    lastSentFan   = fanOn ? 1 : 0;
    lastSentLocked = isLocked ? 1 : 0;

    triggerBeep(150);

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    int cnt=0;
    while (WiFi.status()!=WL_CONNECTED && cnt<30) { delay(500); cnt++; }

    esp_wifi_set_ps(WIFI_PS_NONE);

    if (esp_now_init()!=ESP_OK) { Serial.println("ESP-NOW failed!"); return; }
    esp_now_register_recv_cb(onDataReceived);
    esp_now_register_send_cb(onDataSent);

    esp_now_peer_info_t peer={};
    memcpy(peer.peer_addr, masterMAC, 6);
    peer.channel=0; peer.encrypt=false;
    esp_now_add_peer(&peer);

    sendHello();
    lastHelloTime = millis();
}

```

```

}

// -----
void loop() {
  if (newMsgAvailable) {
    noInterrupts();
    strcpy(processBuf, rxBuffer);
    newMsgAvailable = false;
    interrupts();
    processCommand(String(processBuf));
  }

  if (!masterAked && millis()-lastHelloTime > HELLO_INTERVAL_MS) {
    lastHelloTime=millis(); sendHello();
  }

  motionNow = (digitalRead(IR_PIN)==LOW);
  bool rawDark = (digitalRead(LDR_PIN)==HIGH);
  isDark = (millis()-lastLightOffTime < LDR_RECOVERY_MS) ? true : rawDark;
  if (motionNow) lastMotionTime = millis();

  if (autoMode) {
    if (motionNow && isDark && !lightOn) {
      lastLocalLightChange=millis();
      setLight(true); sendHallStatus();
    }
    else if (!motionNow && lightOn && millis()-lastMotionTime > MOTION_TIMEOUT_MS) {
      lastLocalLightChange=millis();
      setLight(false); sendHallStatus();
    }
  }

  if (millis()-lastStateChangeTime > DEBOUNCE_LOCKOUT_MS) {
    static bool lastTL=false;
    bool tL=(digitalRead(TOUCH_LIGHT_PIN)==HIGH);
    if (tL && !lastTL) {
      lastLocalLightChange=millis();
      triggerBeep(80); setLight(!lightOn); sendHallStatus();
    }
    lastTL=tL;

    static bool lastTF=false;
    bool tF=(digitalRead(TOUCH_FAN_PIN)==HIGH);
    if (tF && !lastTF) {
      lastLocalFanChange=millis();
      triggerBeep(80); setFan(!fanOn); sendHallStatus();
    }
    lastTF=tF;
  }

  static bool lastTD=false;
  bool tD=(digitalRead(DOOR_TOUCH_PIN)==HIGH);
  if (tD && !lastTD && millis()-lastDoorTouchTime > TOUCH_DEBOUNCE_MS) {
    lastDoorTouchTime = millis();
    lastDoorLocalChange = millis();
    bool ns = !isLocked;
    setLock(ns); lastSentLocked=ns?1:0;
    sendDoorStatus(); triggerBeep(80);
  }
  lastTD=tD;
}

```

19.2 Kitchen Slave (ESP32)

```
#include <WiFi.h>
#include <esp_now.h>

#define WIFI_SSID      "AP_302"
#define WIFI_PASSWORD  "[REDACTED]"

#define LIGHT_PIN      45
#define EXHAUST_PIN    48
#define TOUCH_LIGHT_PIN 47
#define TOUCH_EXHST_PIN 21
#define MQ2_DOUT_PIN   13

uint8_t masterMAC[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}; // [REDACTED]

// -- State --
bool lightOn      = false;
bool exhaustOn    = false;
bool gasDetected  = false;
bool buzzerOn     = false;
bool lastGasState = false;

// -- Gas Logic --
int gasConfidence = 0;
#define GAS_THRESHOLD 3 // Reduced to 3 for faster automatic response
unsigned long warmUpFinishedTime = 0;
#define WARM_UP_DELAY_MS 30000 // 30s warmup to prevent false boot alarms

// -- Timers --
unsigned long lastTouchLightTime = 0, lastTouchExhstTime = 0;
#define TOUCH_DEBOUNCE_MS 500
unsigned long bootIgnoreUntil = 0;
#define BOOT_IGNORE_MS 6000
unsigned long lastGasCheck = 0;
#define GAS_CHECK_MS 200
unsigned long lastHelloTime = 0;
#define HELLO_INTERVAL_MS 5000

char outBuf[128];
bool masterAcked = false;

// -----
void setLight(bool on) {
    lightOn = on;
    digitalWrite(LIGHT_PIN, on ? HIGH : LOW);
    Serial.println("Light -> " + String(on ? "ON" : "OFF"));
}

void setExhaust(bool on) {
    exhaustOn = on;
    digitalWrite(EXHAUST_PIN, on ? HIGH : LOW);
    Serial.println("Exhaust -> " + String(on ? "ON" : "OFF"));
}

void sendAlert(String p) {
    String pkt = "ALERT|Kitchen|" + p;
    pkt.toCharArray(outBuf, 128);
    esp_now_send(masterMAC, (uint8_t*)outBuf, 128);
}

void sendFullStatus() {
    sendAlert(
        "LIGHT=" + String(lightOn ? "1" : "0") +
        ";EXHAUST=" + String(exhaustOn ? "1" : "0") +
        ";GAS=" + String(gasDetected ? "1" : "0") +
        ";BUZZER=" + String(buzzerOn ? "1" : "0")
    );
}

// -- ESP-NOW Receive --
```

```

void onDataReceived(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
    String msg = String((char*)data);
    if (!masterAked) { masterAked = true; }

    int p1 = msg.indexOf('|'), p2 = msg.indexOf('|', p1+1);
    if (p1<0||p2<0) return;
    String type = msg.substring(0, p1);
    String room = msg.substring(p1+1, p2);
    String cmd = msg.substring(p2+1);
    if (type!="CMD"||room!="Kitchen") return;

    if (millis() < bootIgnoreUntil) return;

    if (cmd == "LED_ON") { setLight(true); sendFullStatus(); }
    else if (cmd == "LED_OFF") { setLight(false); sendFullStatus(); }
    else if (cmd == "EXHAUST_ON") { setExhaust(true); sendFullStatus(); }
    else if (cmd == "EXHAUST_OFF") {
        // BLOCK "OFF" command if gas is still present
        if (!gasDetected) { setExhaust(false); sendFullStatus(); }
        else { Serial.println("Command Ignored: Gas detected, Exhaust must stay ON"); }
    }
    else if (cmd == "STATUS") { sendFullStatus(); }
}

// -- AUTOMATIC GAS LOGIC --
void checkGas() {
    if (millis() < warmUpFinishedTime) return;

    bool rawGas = (digitalRead(MQ2_DOUT_PIN) == LOW); // LOW means Gas detected

    if (rawGas) {
        if (gasConfidence < GAS_THRESHOLD) gasConfidence++;
    } else {
        if (gasConfidence > 0) gasConfidence--;
    }

    bool currentGasStatus = (gasConfidence >= GAS_THRESHOLD);

    if (currentGasStatus != lastGasState) {
        lastGasState = currentGasStatus;
        gasDetected = currentGasStatus;

        if (gasDetected) {
            // GAS DETECTED - ACTING AUTOMATICALLY
            buzzerOn = true;
            setExhaust(true); // <--- AUTOMATIC ON
            Serial.println("GAS DETECTED! Exhaust turned ON automatically.");
            sendFullStatus();
        } else {
            // GAS CLEARED - ACTING AUTOMATICALLY
            buzzerOn = false;
            setExhaust(false); // <--- AUTOMATIC OFF
            Serial.println("GAS CLEARED. Exhaust turned OFF automatically.");
            sendFullStatus();
        }
    }
}

void setup() {
    Serial.begin(115200);
    bootIgnoreUntil = millis() + BOOT_IGNORE_MS;
    warmUpFinishedTime = millis() + WARM_UP_DELAY_MS;

    pinMode(LIGHT_PIN, OUTPUT);
    pinMode(EXHAUST_PIN, OUTPUT);
    pinMode(TOUCH_LIGHT_PIN, INPUT);
    pinMode(TOUCH_EXHST_PIN, INPUT);
    pinMode(MQ2_DOUT_PIN, INPUT_PULLUP);

    setLight(false);
    setExhaust(false);
}

```

```

WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
while (WiFi.status()!=WL_CONNECTED) { delay(500); }

if (esp_now_init()==ESP_OK) {
  esp_now_register_recv_cb(onDataReceived);
  esp_now_peer_info_t mp; memset(&mp,0,sizeof(mp));
  memcpy(mp.peer_addr, masterMAC, 6);
  esp_now_add_peer(&mp);
}

Serial.println("System Ready. MQ-2 Warming up...");
}

void loop() {
  if (!masterAked && millis()-lastHelloTime > HELLO_INTERVAL_MS) {
    lastHelloTime = millis();
    String pkt = "HELLO|Kitchen|";
    esp_now_send(masterMAC, (uint8_t*)pkt.c_str(), pkt.length());
  }

  // 1. Always check gas first for safety
  if (millis() - lastGasCheck > GAS_CHECK_MS) {
    lastGasCheck = millis();
    checkGas();
  }

  // 2. Touch Light
  bool tL = (digitalRead(TOUCH_LIGHT_PIN)==HIGH);
  if (tL && millis()-lastTouchLightTime > TOUCH_DEBOUNCE_MS) {
    lastTouchLightTime = millis();
    setLight(!lightOn);
    sendFullStatus();
  }

  // 3. Touch Exhaust (Blocked if gas is present)
  bool tE = (digitalRead(TOUCH_EXHST_PIN)==HIGH);
  if (tE && millis()-lastTouchExhstTime > TOUCH_DEBOUNCE_MS) {
    lastTouchExhstTime = millis();
    if (!gasDetected) {
      setExhaust(!exhaustOn);
      sendFullStatus();
    } else {
      Serial.println("Touch ignored: Safety lock active (Gas).");
    }
  }
}
}

```

19.3 Bedroom Slave (ESP32-S3)

Note: no separate bedroom hardware photo was provided; wiring follows the pin table in Section 3.2.

```
#include <WiFi.h>
#include <esp_now.h>
#include <Wire.h>
#include <Adafruit_MCP9808.h>

#define WIFI_SSID "Reyy Ra"
#define WIFI_PASSWORD "[REDACTED]"
#define PKT_SIZE 128

// ESP32-S3 Pin Allocations
#define LED_PIN 3
#define FAN_PWM 9
#define TOUCH_LED 10
#define TOUCH_FAN 37
#define PIR_PIN 38 // IR sensor pin
#define SDA_PIN 39
#define SCL_PIN 40

#define PWM_FREQ 25000
#define PWM_RESOLUTION 8
#define FAN_OFF 0
#define FAN_LOW 100
#define FAN_HIGH 255
#define TEMP_HIGH 30.0
#define TEMP_LOW 27.0

// -- IR Sensor Configuration --
#define IR_ACTIVE_STATE LOW

uint8_t masterMAC[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}; // [REDACTED]

// -- State variables --
bool ledState = false;
int fanSpeed = FAN_OFF;
bool autoMode = false;
bool confirmedPIR = false;

// -- Timers & Intervals --
unsigned long bootIgnoreUntil = 0;
#define BOOT_IGNORE_MS 6000

unsigned long lastHelloTime = 0;
#define HELLO_INTERVAL_MS 5000

// IR Sensor Filter Timers
unsigned long pirLowSince = 0;
bool pirLowTimerRunning = false;
#define PIR_CLEAR_MS 3000 // Time to wait before clearing motion
#define IR_DEBOUNCE_MS 150 // Time the IR sensor state must remain stable

// Temperature Cache (Prevents I2C lockups)
float cachedTemp = 25.0;
unsigned long lastTempCheck = 0;
#define TEMP_READ_INTERVAL_MS 4000

// ESP-NOW Safe Transmission Queue
unsigned long lastStatusSent = 0;
#define STATUS_MIN_INTERVAL_MS 200
bool pendingStatusSend = false;

char outBuf[PKT_SIZE];
bool masterAked = false;

Adafruit_MCP9808 tempSensor;
bool mcpFound = false;
```

```

// Forward Declarations
void sendFullStatus();
void sendFullStatusInstant();
void updateTemperatureCached();
void processPendingStatus();

// -- Hardware Control --
void setLED(bool on) {
    ledState = on;
    digitalWrite(LED_PIN, on ? HIGH : LOW);
    Serial.println("LED -> " + String(on ? "ON" : "OFF"));
}

void setFanPWM(int speed) {
    fanSpeed = speed;
    ledcWrite(FAN_PWM, speed);
    String level = (speed == FAN_OFF) ? "OFF" : (speed == FAN_LOW) ? "LOW" : "HIGH";
    Serial.println("Fan -> " + level);
}

void updateTemperatureCached() {
    if (!mcpFound) return;
    unsigned long now = millis();
    if (now - lastTempCheck >= TEMP_READ_INTERVAL_MS || lastTempCheck == 0) {
        lastTempCheck = now;
        tempSensor.wake();
        float t = tempSensor.readTempC();
        tempSensor.shutdown_wake(1);
        if (t > -10.0 && t < 60.0) { // Keep only sane readings
            cachedTemp = t;
        }
    }
}

// Turns off appliances but does not alter the autoMode state
void autoShutdown() {
    bool changed = false;
    if (ledState) {
        setLED(false);
        changed = true;
    }
    if (fanSpeed != FAN_OFF) {
        setFanPWM(FAN_OFF);
        changed = true;
    }
    if (changed) {
        sendFullStatus();
    }
}

// -- Status Updates (Kitchen Semicolon Format) --
void sendAlert(String p) {
    String pkt = "ALERT|Bedroom|" + p;
    pkt.toCharArray(outBuf, PKT_SIZE);
    esp_now_send(masterMAC, (uint8_t*)outBuf, PKT_SIZE);
}

// Queue status for sending (used for continuous loop transitions)
void sendFullStatus() {
    pendingStatusSend = true;
}

// Sends status immediately (used for manual operations to keep UI in sync)
void sendFullStatusInstant() {
    unsigned long now = millis();
    lastStatusSent = now;
    pendingStatusSend = false; // Clear any pending queued updates

    String fanLevel = (fanSpeed == FAN_OFF) ? "0" : (fanSpeed == FAN_LOW) ? "1" : "2";
    sendAlert(
        "LIGHT=" + String(ledState ? "1" : "0") +
        ";FAN=" + fanLevel +
        ";AUTO=" + String(autoMode ? "1" : "0") +
        ";TEMP=" + (mcpFound ? String(cachedTemp, 1) : "error") +
    );
}

```

```

    ";PIR=" + String(confirmedPIR ? "1" : "0")
  );
}

void processPendingStatus() {
  unsigned long now = millis();
  if (pendingStatusSend && (now - lastStatusSent >= STATUS_MIN_INTERVAL_MS)) {
    pendingStatusSend = false;
    lastStatusSent = now;

    String fanLevel = (fanSpeed == FAN_OFF) ? "0" : (fanSpeed == FAN_LOW) ? "1" : "2";
    sendAlert(
      "LIGHT=" + String(ledState ? "1" : "0") +
      ";FAN=" + fanLevel +
      ";AUTO=" + String(autoMode ? "1" : "0") +
      ";TEMP=" + (mcpFound ? String(cachedTemp, 1) : "error") +
      ";PIR=" + String(confirmedPIR ? "1" : "0")
    );
  }
}

// -- ESP-NOW Receive --
void onDataReceived(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
  char tempBuf[PKT_SIZE];
  int copyLen = (len < PKT_SIZE) ? len : (PKT_SIZE - 1);
  memcpy(tempBuf, data, copyLen);
  tempBuf[copyLen] = '\0';

  String msg = String(tempBuf);
  if (!masterAked) {
    masterAked = true;
  }

  int p1 = msg.indexOf('|'), p2 = msg.indexOf('|', p1+1);
  if (p1 < 0 || p2 < 0) return;
  String type = msg.substring(0, p1);
  String room = msg.substring(p1+1, p2);
  String cmd = msg.substring(p2+1);

  if (type != "CMD" || room != "Bedroom") return;
  if (millis() < bootIgnoreUntil) return;

  // Manual commands do not reset autoMode to false.
  if (cmd == "LED_ON") {
    setLED(true);
    sendFullStatusInstant();
  }
  else if (cmd == "LED_OFF") {
    setLED(false);
    sendFullStatusInstant();
  }
  else if (cmd == "FAN_ON" || cmd == "FAN_LOW") {
    setFanPWM(FAN_LOW);
    sendFullStatusInstant();
  }
  else if (cmd == "FAN_HIGH") {
    setFanPWM(FAN_HIGH);
    sendFullStatusInstant();
  }
  else if (cmd == "FAN_OFF") {
    setFanPWM(FAN_OFF);
    sendFullStatusInstant();
  }
  else if (cmd == "AUTO_ON") {
    autoMode = true;
    if (!confirmedPIR) {
      autoShutdown(); // Immediate initial shutdown if empty on activation
    }
    sendFullStatusInstant();
  }
  else if (cmd == "AUTO_OFF") {
    autoMode = false;
    sendFullStatusInstant();
  }
}

```

```

    else if (cmd == "STATUS") {
        sendFullStatusInstant();
    }
}

void setup() {
    Serial.begin(115200);
    bootIgnoreUntil = millis() + BOOT_IGNORE_MS;

    pinMode(LED_PIN, OUTPUT);
    setLED(false);

    pinMode(PIR_PIN, INPUT);

    // Touch pads configured with pull-downs
    pinMode(TOUCH_LED, INPUT_PULLDOWN);
    pinMode(TOUCH_FAN, INPUT_PULLDOWN);

    ledcAttach(FAN_PWM, PWM_FREQ, PWM_RESOLUTION);
    ledcWrite(FAN_PWM, FAN_OFF);

    // Configure I2C with hardware safety timeouts
    Wire.begin(SDA_PIN, SCL_PIN);
    Wire.setTimeout(100);

    if (tempSensor.begin(0x18)) {
        mcpFound = true;
        tempSensor.setResolution(3);
        Serial.println("MCP9808 found!");
    } else {
        Serial.println("MCP9808 NOT found!");
    }

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
    }

    if (esp_now_init() == ESP_OK) {
        esp_now_register_recv_cb(onDataReceived);
        esp_now_peer_info_t mp;
        memset(&mp, 0, sizeof(mp));
        memcpy(mp.peer_addr, masterMAC, 6);
        esp_now_add_peer(&mp);
    }

    Serial.println("Bedroom ESP32-S3 System Ready.");
}

void loop() {
    unsigned long now = millis();

    // Update temperature reading periodically in the background
    updateTemperatureCached();

    // Handshake loop
    if (!masterAked && now - lastHelloTime > HELLO_INTERVAL_MS) {
        lastHelloTime = now;
        String pkt = "HELLO|Bedroom|";
        esp_now_send(masterMAC, (uint8_t*)pkt.c_str(), pkt.length());
    }

    bool booting = (now < bootIgnoreUntil);

    if (!booting) {
        // 1. Debounced IR Sensor State Processing (Edge-Triggered Logic)
        bool rawSensorState = (digitalRead(PIR_PIN) == IR_ACTIVE_STATE);
        static bool lastRawSensorState = false;
        static unsigned long lastRawStateChange = 0;
    }
}

```

```

if (rawSensorState != lastRawSensorState) {
    lastRawSensorState = rawSensorState;
    lastRawStateChange = now;
}

if (now - lastRawStateChange > IR_DEBOUNCE_MS) {
    if (rawSensorState) {
        // Confirmed Motion Detected
        pirLowTimerRunning = false;
        if (!confirmedPIR) {
            confirmedPIR = true;
            Serial.println("Motion Detected!");
            sendFullStatus();
        }
    } else {
        // Confirmed Motion Cleared
        if (confirmedPIR) {
            if (!pirLowTimerRunning) {
                pirLowTimerRunning = true;
                pirLowSince = now;
            } else if (now - pirLowSince >= PIR_CLEAR_MS) {
                confirmedPIR = false;
                Serial.println("Motion Cleared");
                if (autoMode) {
                    autoShutdown(); // Edge-Triggered Shutdown: Runs once when empty
                } else {
                    sendFullStatus();
                }
            }
            pirLowTimerRunning = false;
        }
    }
}
}

// 2. Stable State-Machine Debouncer for TOUCH_LED
static bool debouncedTouchLED = false;
static bool lastRawTouchLED = false;
static unsigned long lastTouchLEDTime = 0;
#define DEBOUNCE_DELAY_MS 50

bool rawTouchLED = (digitalRead(TOUCH_LED) == HIGH);
if (rawTouchLED != lastRawTouchLED) {
    lastRawTouchLED = rawTouchLED;
    lastTouchLEDTime = now;
}

if ((now - lastTouchLEDTime) > DEBOUNCE_DELAY_MS) {
    if (rawTouchLED != debouncedTouchLED) {
        debouncedTouchLED = rawTouchLED;
        if (debouncedTouchLED) { // Triggers exactly once on rising edge (touch press)
            setLED(!ledState);
            sendFullStatusInstant();
        }
    }
}

// 3. Stable State-Machine Debouncer for TOUCH_FAN
static bool debouncedTouchFAN = false;
static bool lastRawTouchFAN = false;
static unsigned long lastTouchFANTime = 0;

bool rawTouchFAN = (digitalRead(TOUCH_FAN) == HIGH);
if (rawTouchFAN != lastRawTouchFAN) {
    lastRawTouchFAN = rawTouchFAN;
    lastTouchFANTime = now;
}

if ((now - lastTouchFANTime) > DEBOUNCE_DELAY_MS) {
    if (rawTouchFAN != debouncedTouchFAN) {
        debouncedTouchFAN = rawTouchFAN;
        if (debouncedTouchFAN) { // Triggers exactly once on rising edge (touch press)
            setFanPWM(fanSpeed == FAN_OFF ? FAN_LOW : FAN_OFF);
            sendFullStatusInstant();
        }
    }
}

```

```

    }
  }
}

// 4. Auto Mode Dynamic Temp Adjustments
if (autoMode && confirmedPIR) {
  if (fanSpeed != FAN_OFF && (now - lastTempCheck > 3000)) {
    if (cachedTemp >= TEMP_HIGH && fanSpeed != FAN_HIGH) {
      setFanPWM(FAN_HIGH);
      sendFullStatus();
    } else if (cachedTemp < TEMP_LOW && fanSpeed != FAN_LOW) {
      setFanPWM(FAN_LOW);
      sendFullStatus();
    }
  }
}
}

// Handle status updates safely
processPendingStatus();

delay(20);
}

```

20. Conclusion

This home automation system successfully demonstrates a robust, scalable, and feature-rich solution for modern smart homes. The master-slave architecture using ESP-NOW provides reliable communication, while Firebase cloud integration enables remote monitoring and control. The system's safety features, automated controls, and user-friendly interfaces make it suitable for real-world deployment.

Key Achievements:

- Successful multi-room automation
- Reliable ESP-NOW communication
- Real-time Firebase integration
- Safety protocols implementation
- State persistence
- Scalable architecture